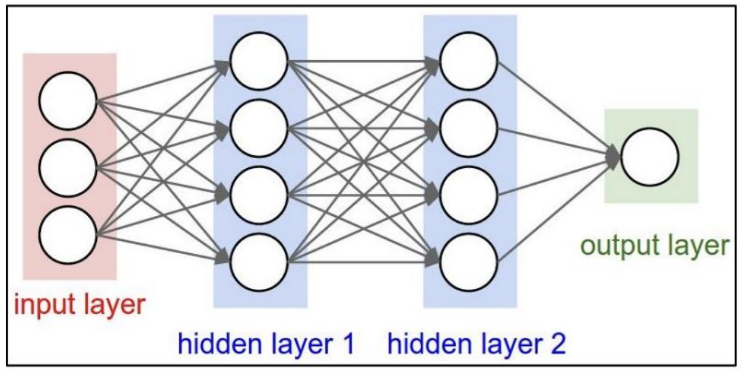
STEVENS
INSTITUTE OF TECHNOLOGY
1870

PEP 559
Machine Learning in Quantum Physics

Dr. Chunlei Qu

Spring 2026

Recap

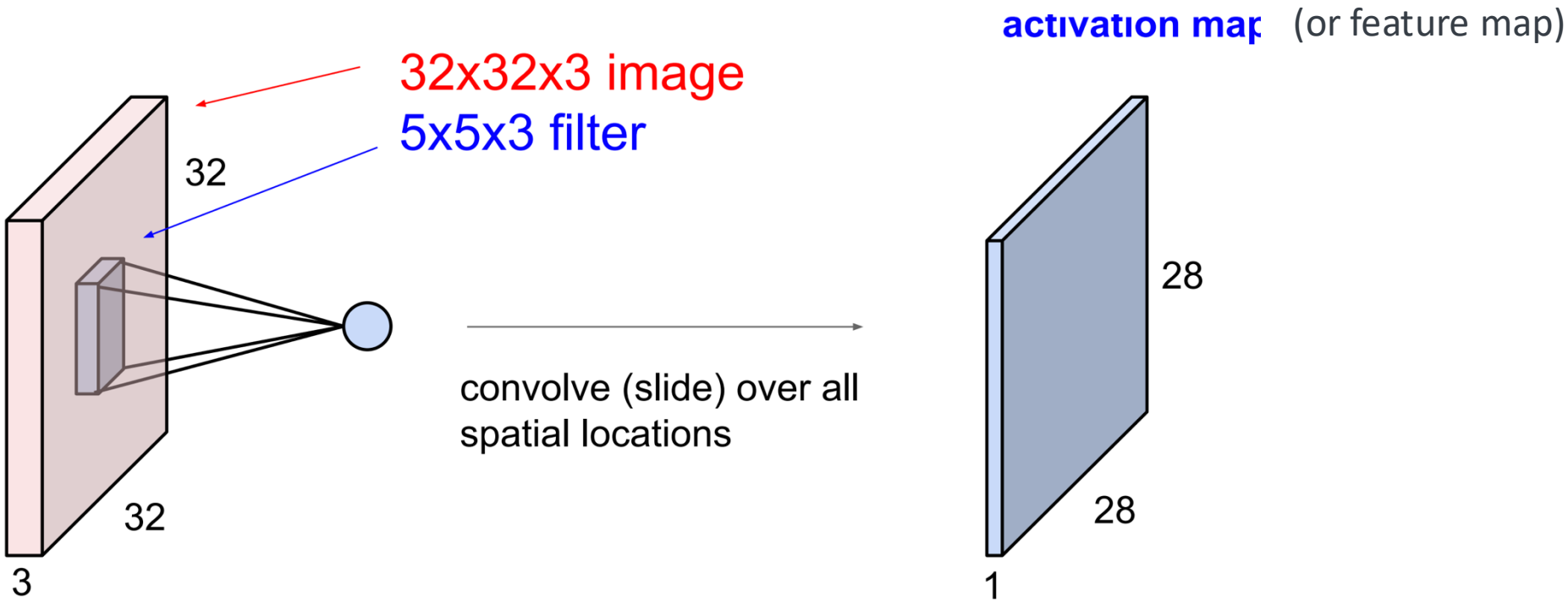


MLP

CNN

- One-hot encoding (labels)
- Feature standardization (input data)
- Activation functions (Step, Sigmoid, softmax)
- Loss functions (MSE, cross-entropy (more later))
- Forward propagation
- Backpropagation
- Stochastic gradient descent (SGD) / mini-batch training
- Padding
- Subsampling /Pooling
- Parameter sharing (CNN filters)

Parameter sharing in CNN

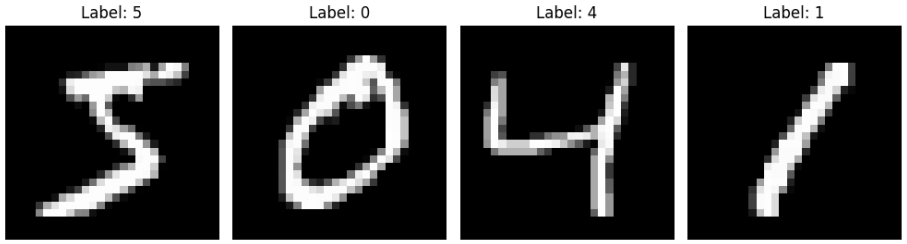


The same filter weights are reused across different locations of the image to detect similar patterns (like an edge or texture which can appear anywhere in the image).

Parameter sharing

- Why is it powerful?

Massive parameter reduction



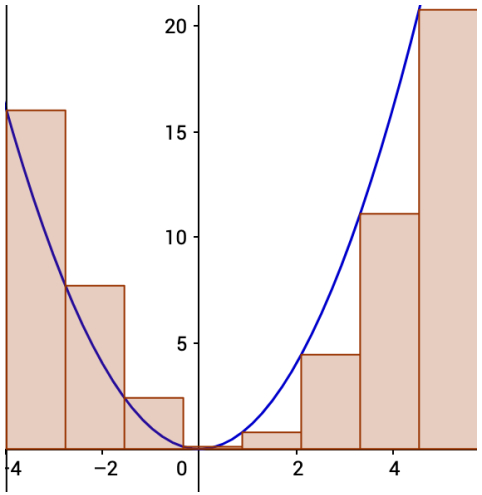
- For MNIST dataset (28*28=784 grayscale image):
  - ➢ if we use an MLP (784 neurons in input layer, 100 neurons in hidden layer, 10 neurons in output layer);
  - ➢ The total number of parameters would be 784*100+100*10=79400 weights (+ biases)
  - ➢ If we use CNN (Conv layer, e.g., 8 filters of size 5x5), the total relevant weights are only 5x5x8=200 parameters
  - ➢ The activation map (if no padding) has (28-5+1)^2*8=4608. After pooling (2x2): (0.5*(28-5+1))^2*8=1152
  - ➢ Then fully connected layer 1152*10=11520 weights
  - ➢ In total, 200+11520=**11720 (CNN) << 79400 (MLP)**

Why not just use 10 hidden neurons in the MLP?

Then the total number of weight will be

784*10 + 10*10 = 7940

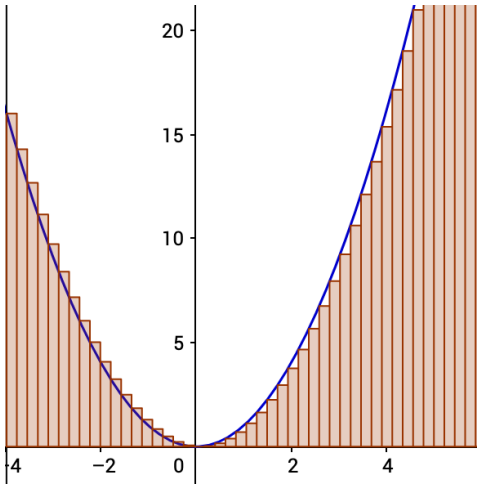
Which is much smaller than the CNN



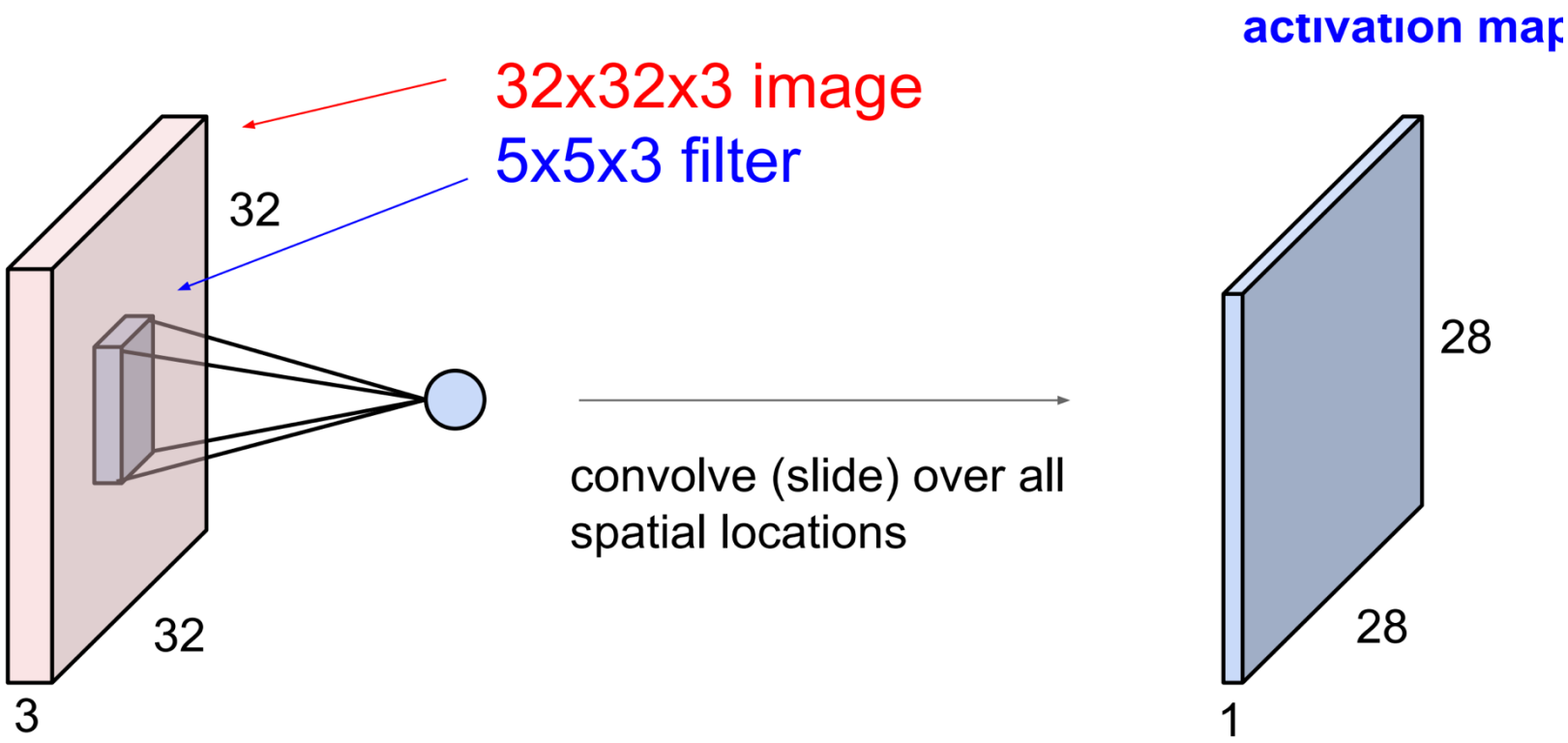
It would not have enough expressive power

Reducing neurons drastically causes

- Severe information compression (784 pixels → 10 features)
- Loss of spatial detail
- Underfitting



Parameter sharing



CNN reduces parameters by sharing the same filter weights across different spatial locations to detect similar local patterns

Recurrent Neural Networks

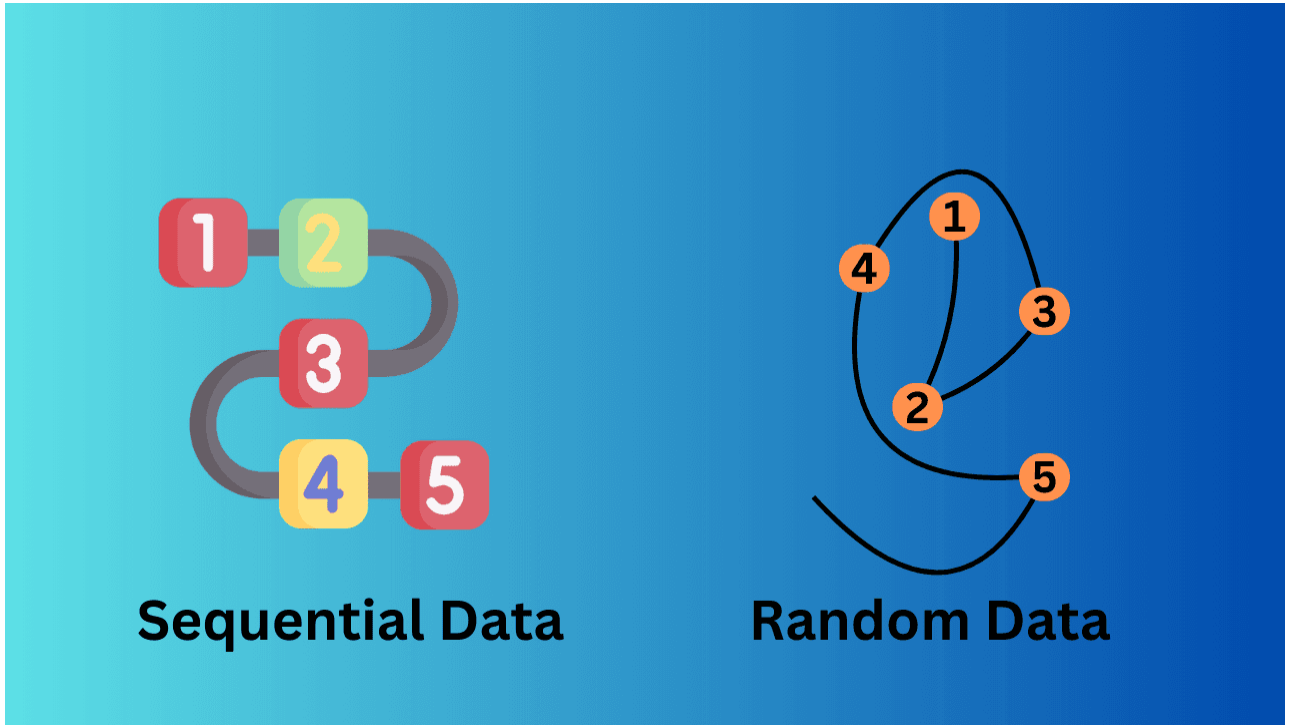
CNN: share weights across space to detect repeated spatial patterns.

RNN: share weights across time to model temporal dependences.

RNNs are particularly useful for sequential (e.g., time-series such as stock prices) data.

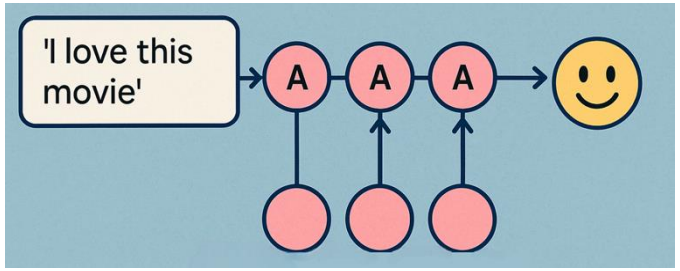
Sequential data

- Order matters
- Time series data is a special type of sequential data (stock prices, voices or speeches).
- Some sequential data do not have explicit time dimension. e.g., text data, DNA sequences



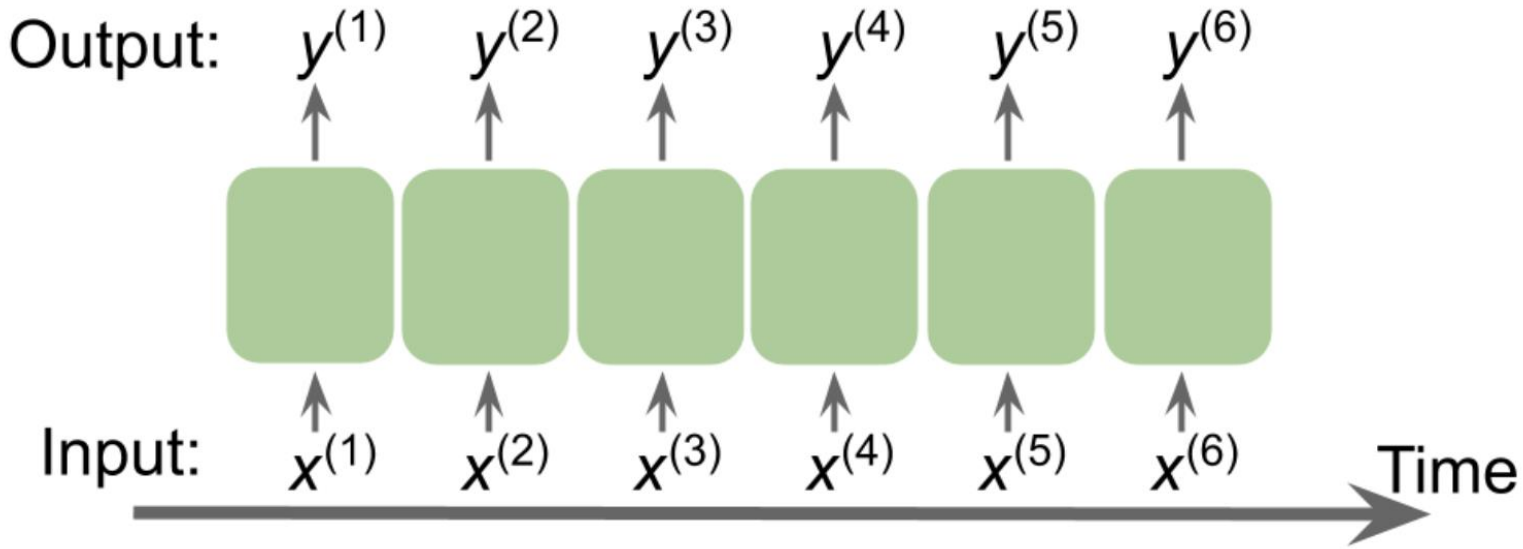
Example: Movie Review

What is the overall sentiment of the review: “good” or “bad”?



Representing sequencies

Consider some time series data of length T. The superscript index indicates the order, t=1, 2, .., T.



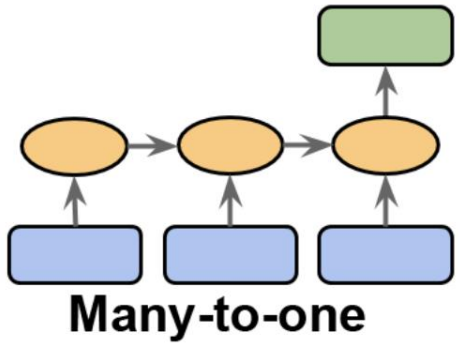
The output and input sizes may not be the same

Most common sequencing tasks

Sentiment analysis

Input: text-based movie review

Output: a label like/dislike



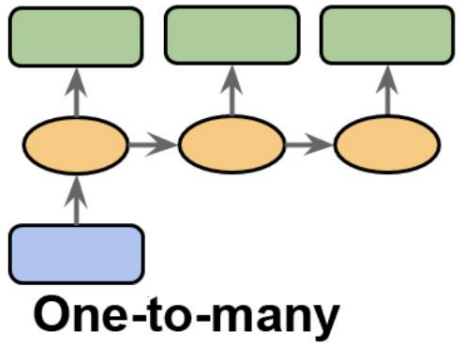
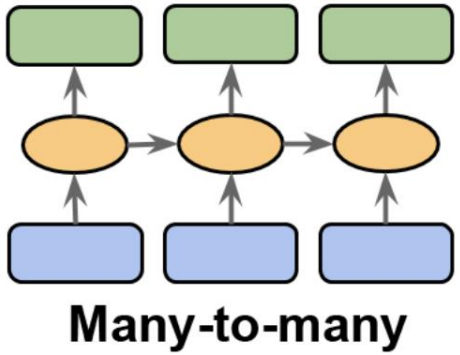
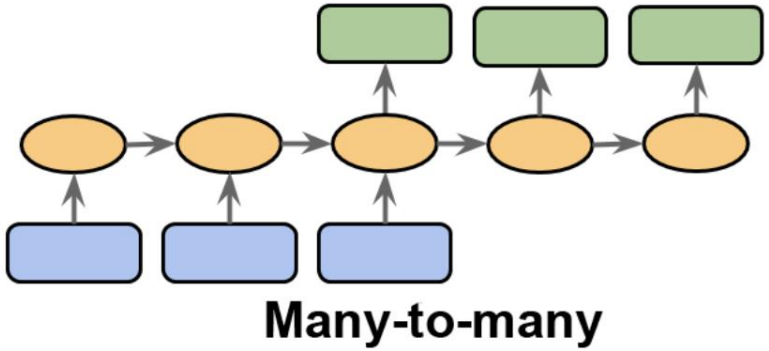


Image captioning

Input: an image

Output: a sentence summarizing the content of the image

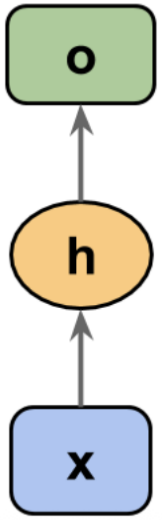




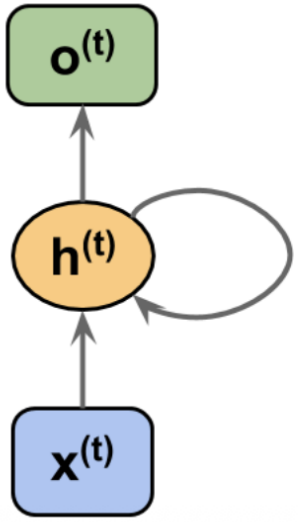
Portfolio optimization (synchronized many-to-many): prices of several stock prices → optimal weights to form a portfolio
Language translation (delayed many-to-many): an English sentence → the German translation

Dataflow of an RNN

A standard feedforward network



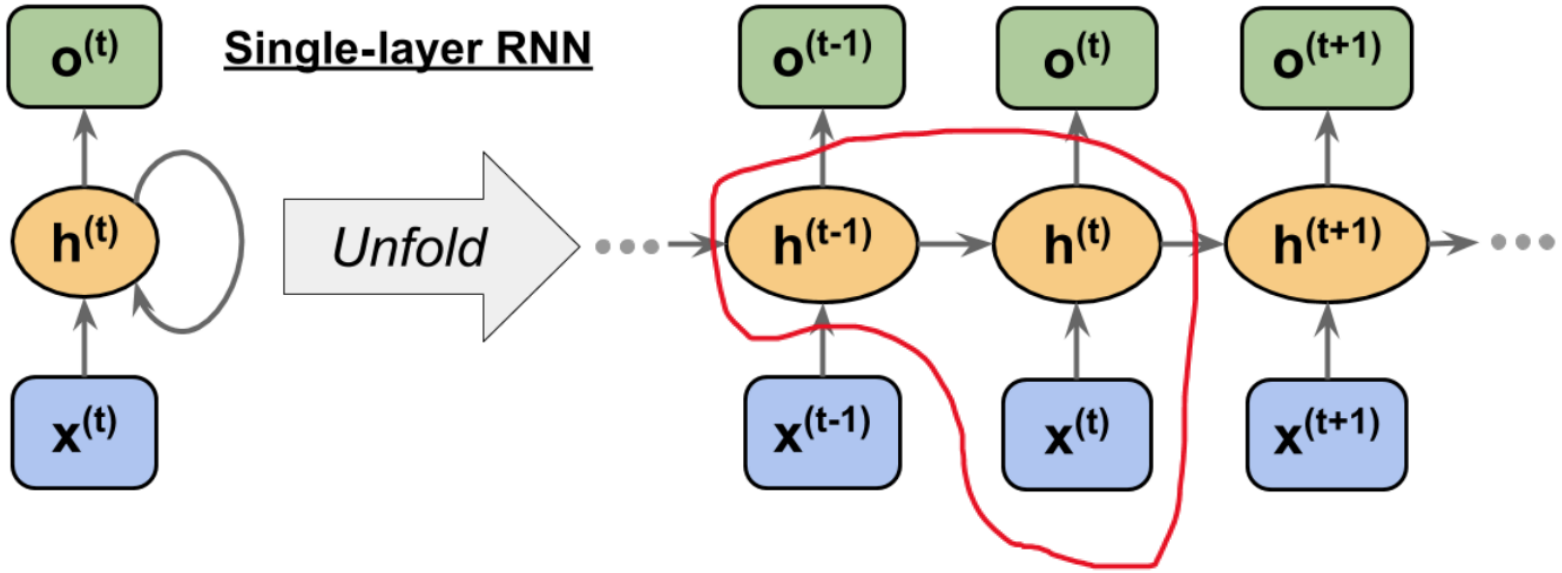
Recurrent neural network



The **loop** (aka **recurrent edge**) allows the network to have a memory of the past events

- In standard feedforward network, information → input layer → hidden layer → output layer
- In an RNN, the hidden layer receives its input from both the input layer of **the current time step** and the hidden layer from **the previous time step**

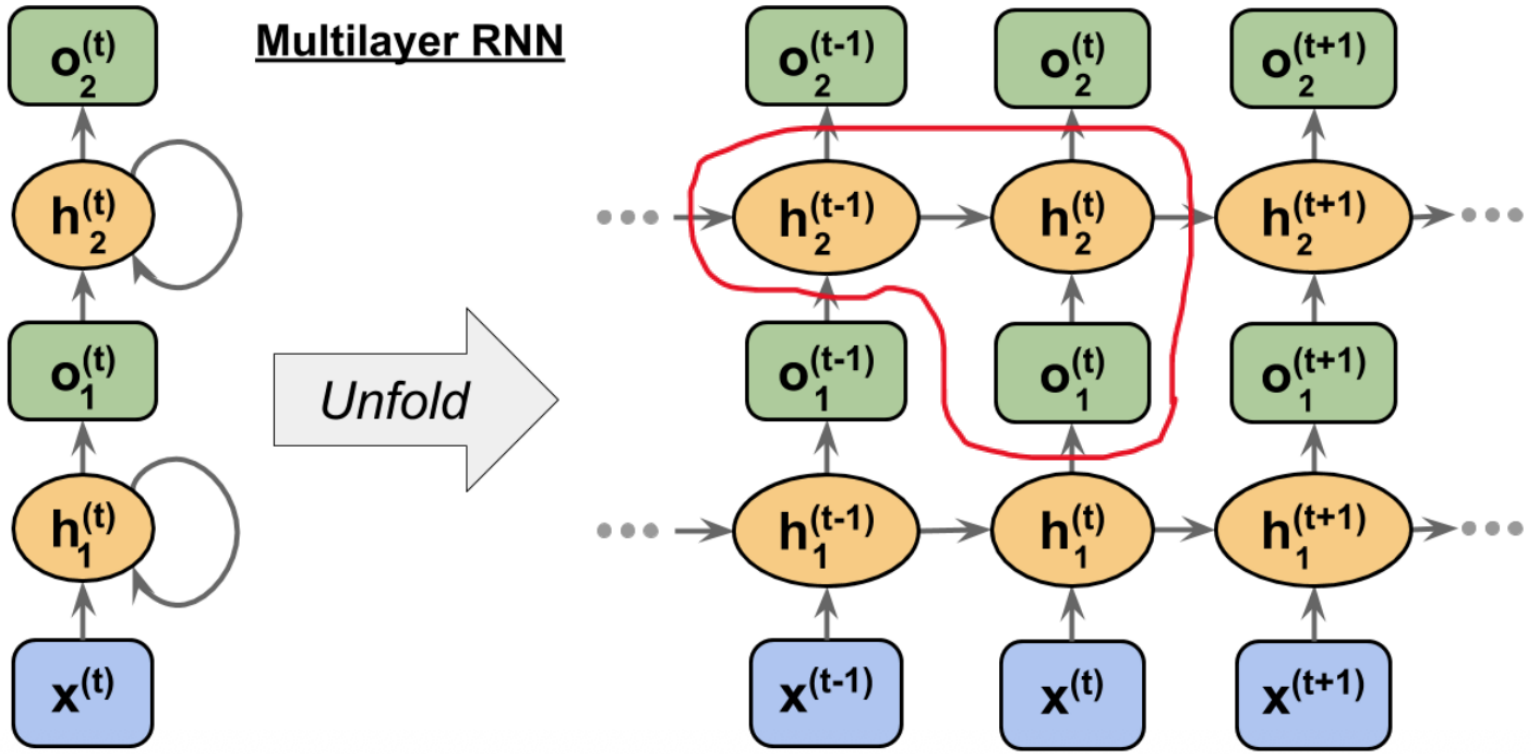
An RNN with one hidden layer



The hidden layer neuron at time t, receives its input from

- The data point at time step, t
- The hidden value at the previous time step, t-1

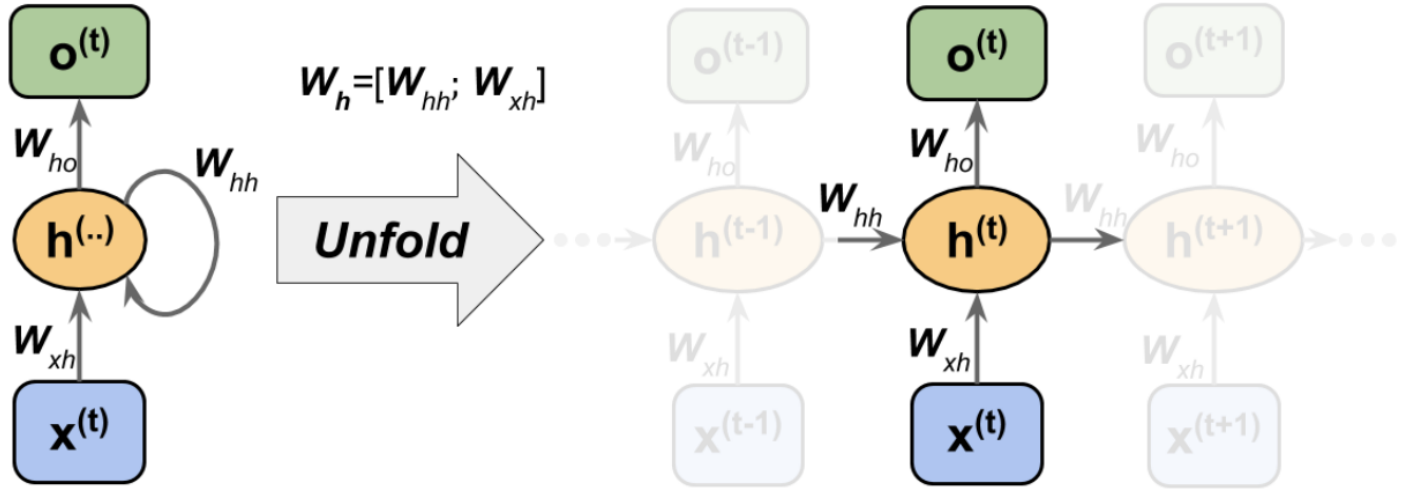
An RNN with two hidden layers



The hidden layer neuron at step (t) receives its input from

- output of the previous layer at the current time step (t) and
- its own hidden value from the previous time step (t-1)

Assigning weights for each directed edge



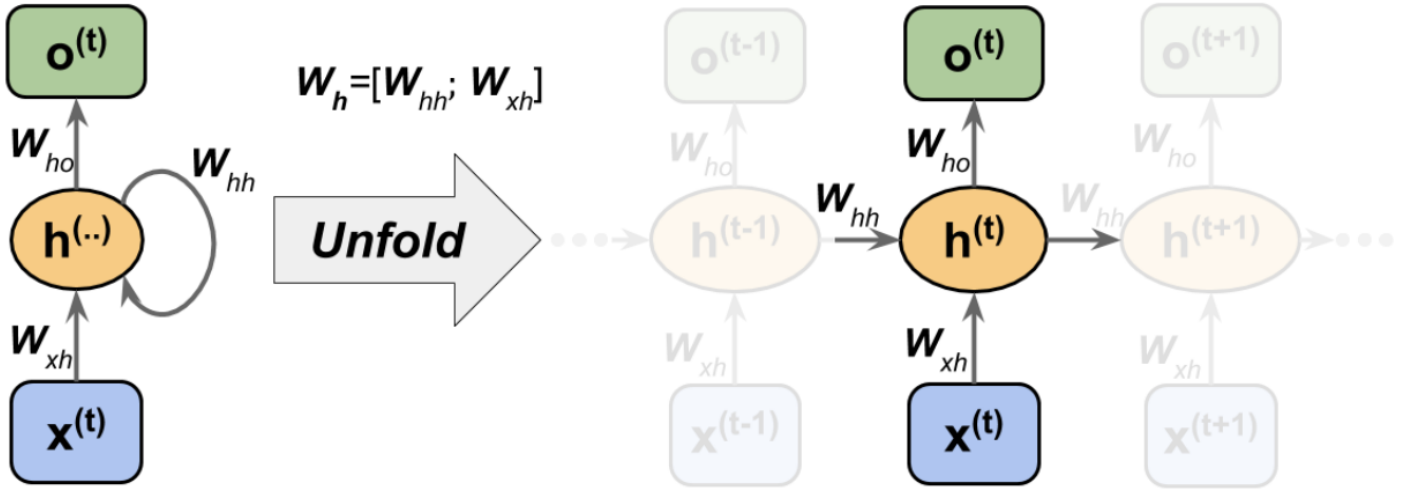
W_{xh}: The weight matrix between the input, $x^{(t)}$, and the hidden layer, h

W_{hh}: The weight matrix associated with the recurrent edge

W_{ho}: The weight matrix between the hidden layer and output layer

Those weights do not depend on time; therefore, they are shared across time

Assigning weights for each directed edge



The pre-activation value of the neuron in hidden layer

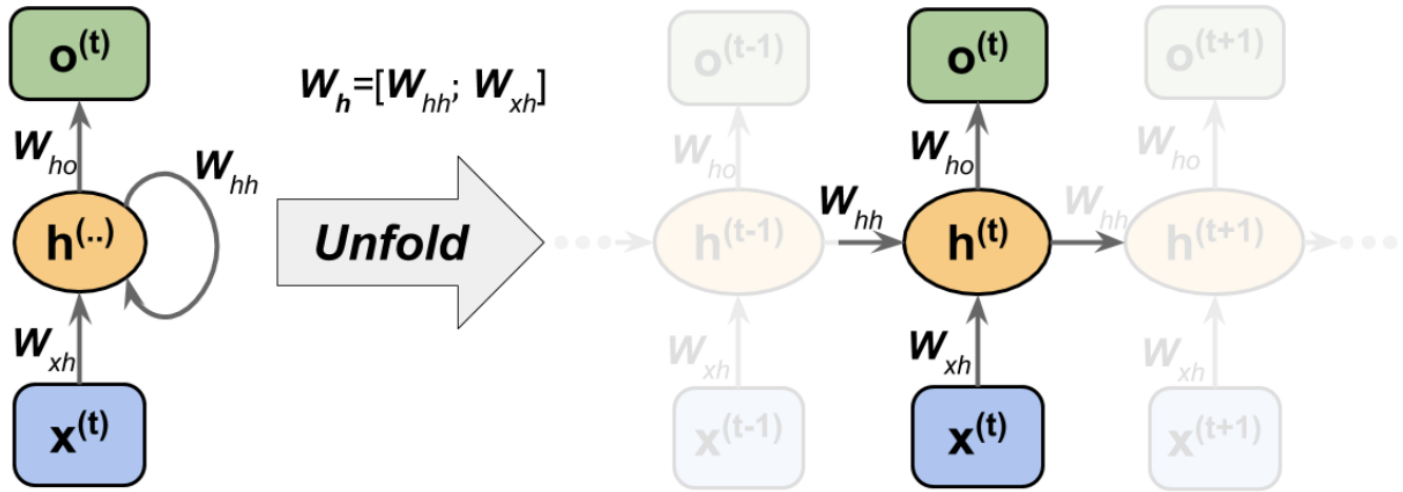
$$\boldsymbol{z}_h^{(t)} = \boldsymbol{W}_{xh}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h$$

After nonlinear activation

$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{z}_h^{(t)}\right)$$

Those weights do not depend on time; therefore, they are shared across time

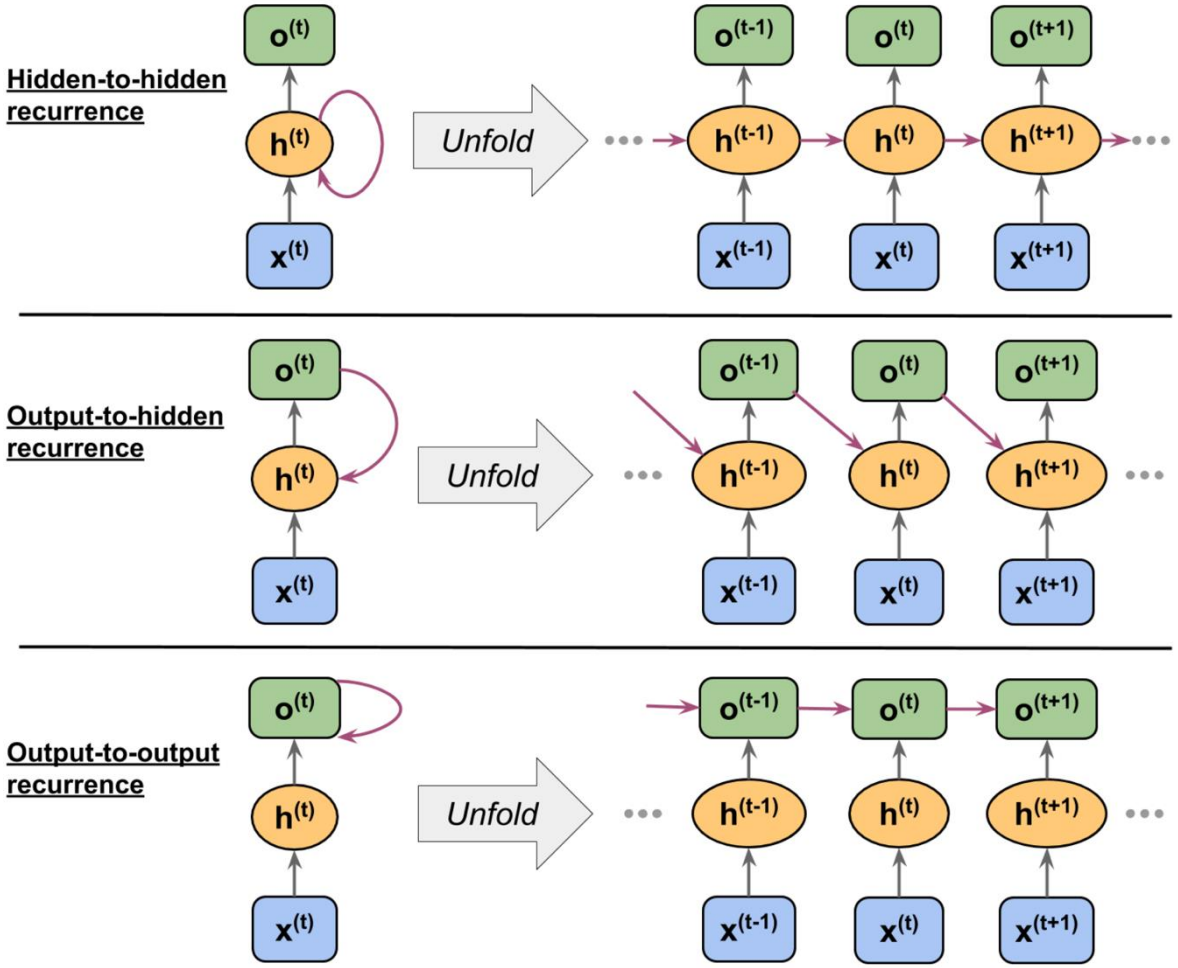
More compact notation



Using concatenated weight matrix

$$\boldsymbol{h}^{(t)} = \sigma_h\left([\boldsymbol{W}_{xh}; \boldsymbol{W}_{hh}]\begin{bmatrix}\boldsymbol{x}^{(t)} \\ \boldsymbol{h}^{(t-1)}\end{bmatrix} + \boldsymbol{b}_h\right) \qquad \boldsymbol{o}^{(t)} = \sigma_0\left(\boldsymbol{W}_{ho}\boldsymbol{h}^{(t)} + \boldsymbol{b}_0\right)$$

Different recurrent connection models



Hidden-to-hidden recurrence

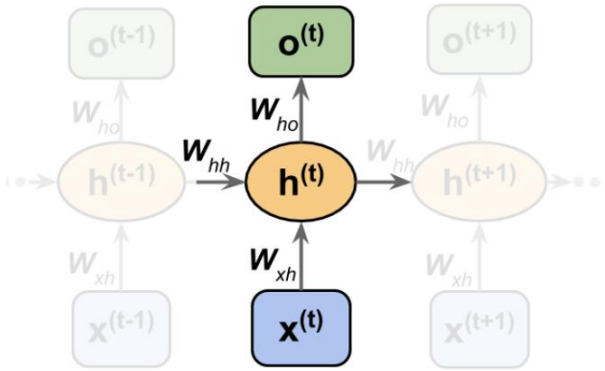
Output-to-hidden recurrence

Output-to-output recurrence

Backpropagation

The overall loss is the sum of all the loss functions at times t=1 to t=T

$$L = \sum_{t=1}^{T} L^{(t)}$$



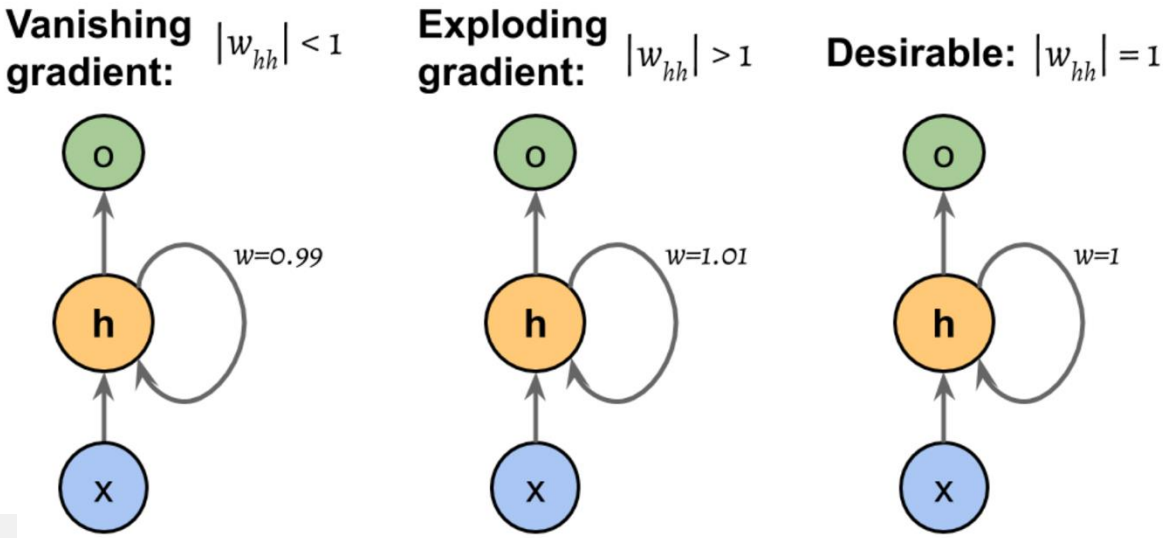
The gradient at time t depends on the hidden units at all previous time steps

$$\frac{\partial L^{(t)}}{\partial \boldsymbol{W}_{hh}} = \frac{\partial L^{(t)}}{\partial \boldsymbol{o}^{(t)}} \times \frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}} \times \left(\sum_{k=1}^{t} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} \times \frac{\partial \boldsymbol{h}^{(k)}}{\partial \boldsymbol{W}_{hh}} \right)$$

Here, $\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}}$ is computed as a multiplication of adjacent time steps:

$$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} = \prod_{i=k+1}^{t} \frac{\partial \boldsymbol{h}^{(i)}}{\partial \boldsymbol{h}^{(i-1)}}$$

Challenges: vanishing and exploding gradients



$$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} = \prod_{i=k+1}^{t} \frac{\partial \boldsymbol{h}^{(i)}}{\partial \boldsymbol{h}^{(i-1)}}$$

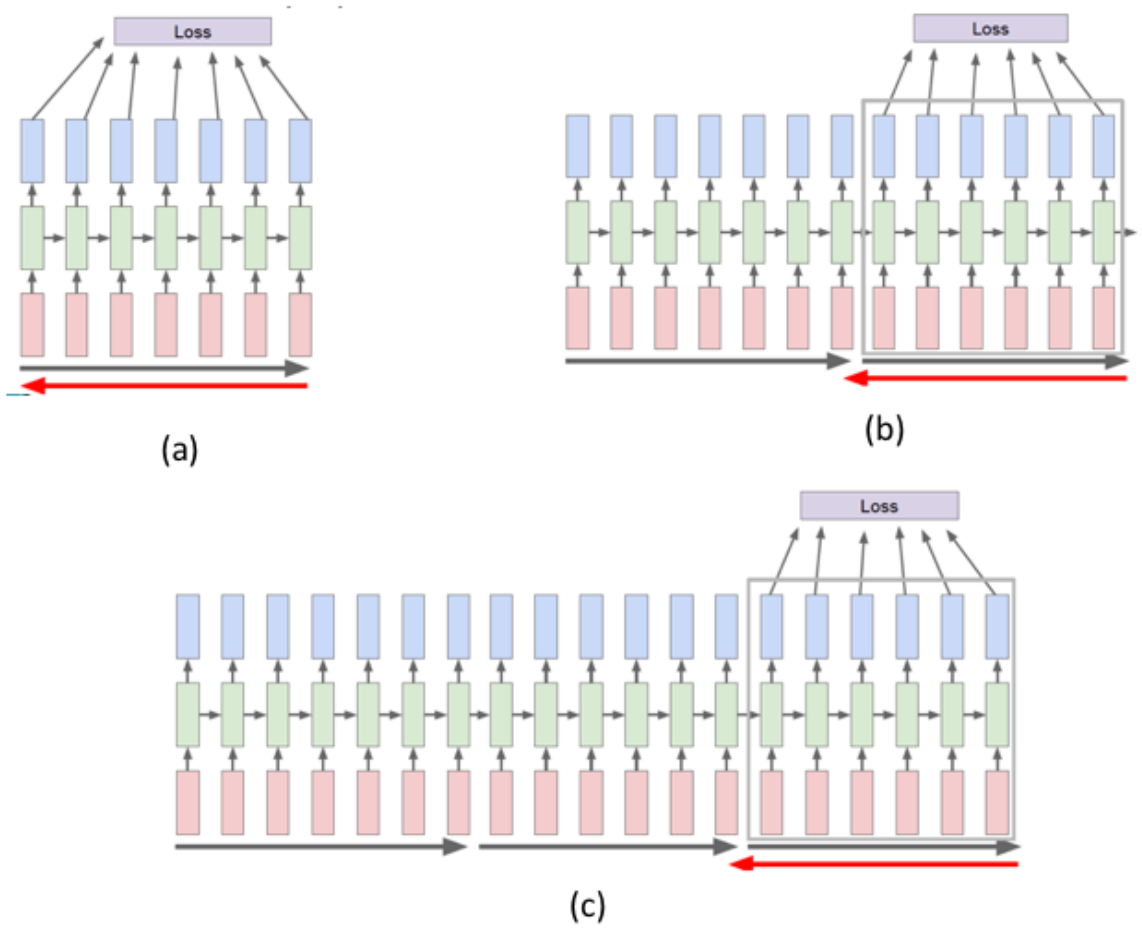
$$\boldsymbol{z}_h^{(t)} = \boldsymbol{W}_{xh}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h$$

$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{z}_h^{(t)}\right)$$

This is essentially to multiply the weight by itself t-k times

Depending on its magnitude, it can be very small or large

Truncated backpropagation through time (TBPTT)



Idea: Backpropagation only a limited number of steps

Break up the input training sequence into smaller segments

(a) BPTT is applied to the 1st segment

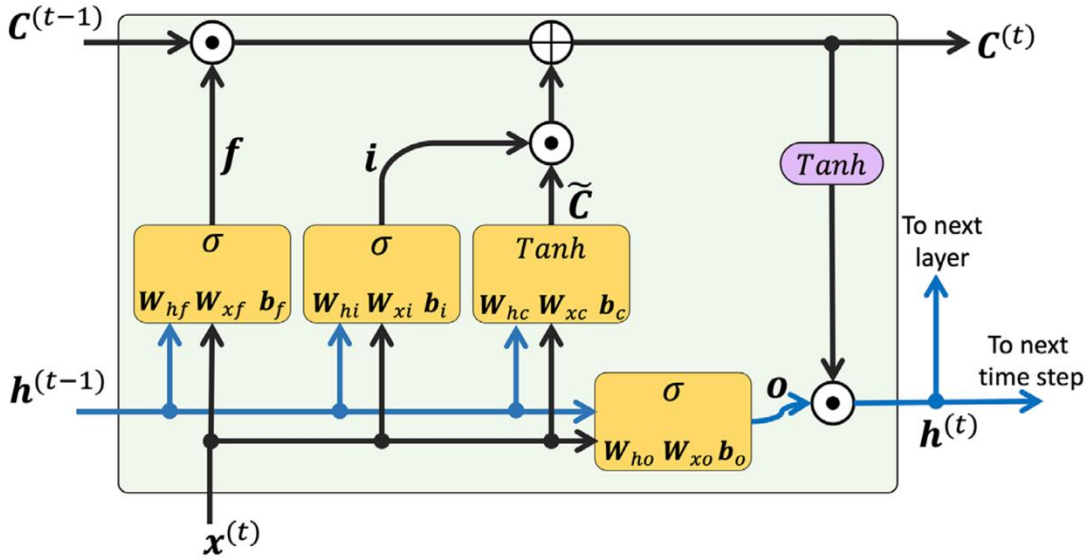
(b) BPTT is applied to the 2nd segment

(c) BPTT is applied to the 3rd segments

TBPTT lets the hidden state carry long-term information forward, but only updates weights using a short recent history

LSTM (Long Short-Term Memory)

- 1997 by Sepp Hochreiter and Jurgen Schmidhuber
- More successful in vanishing and exploding gradient problems

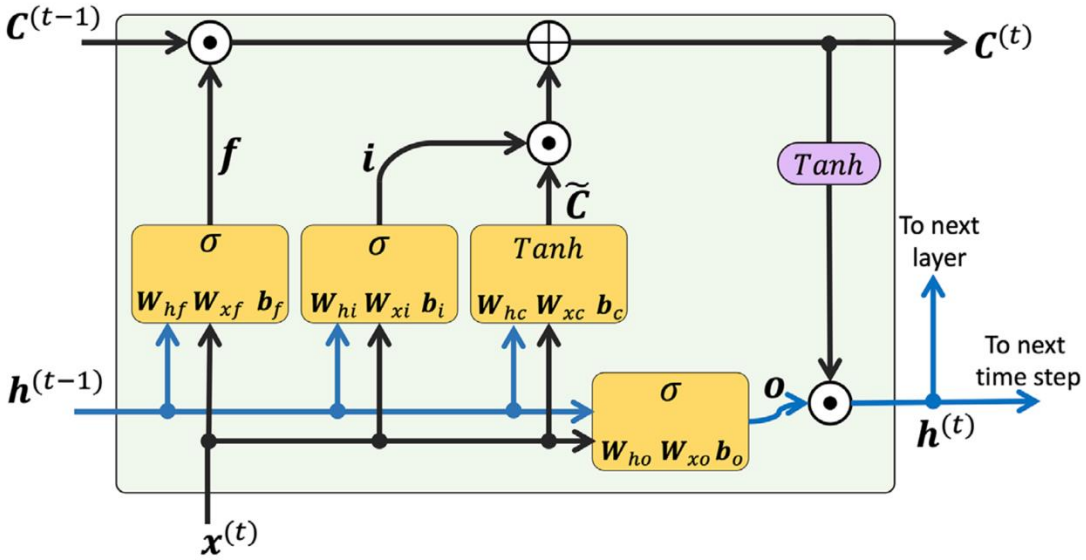


Several computational units

- Forget gate
- Input gate
- Candidate value
- Output gate

Memory cell: Building block of an LSTM

Forget gate

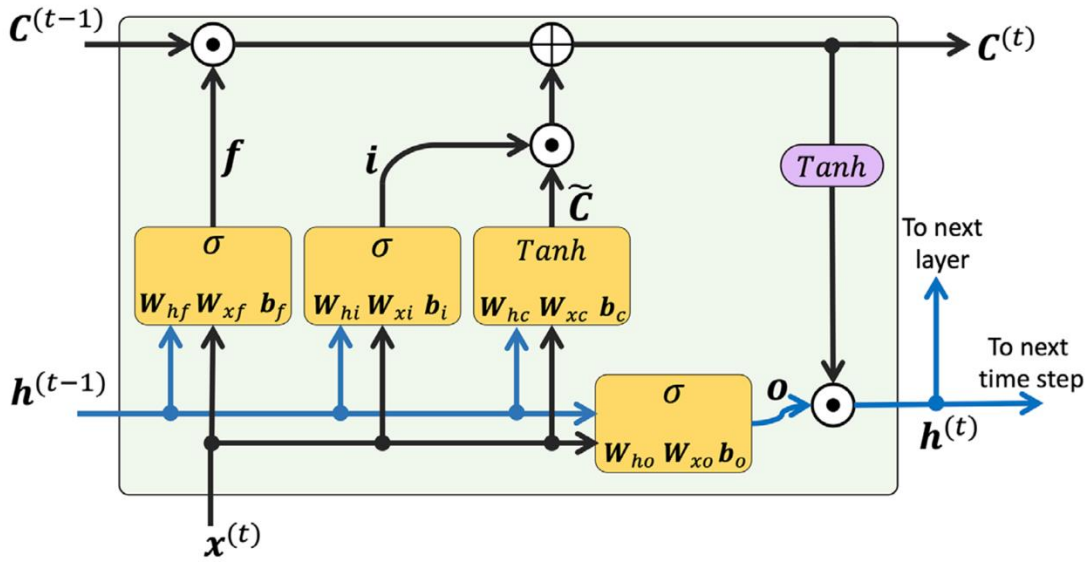


It allows the memory cell to reset the cell state without growing indefinitely

It decides with information is allowed to go through and which information to suppress

$$f_t = \sigma\left(W_{xf}x^{(t)} + W_{hf}h^{(t-1)} + b_f\right)$$

Input gate and candidate value



They are responsible for updating the cell state

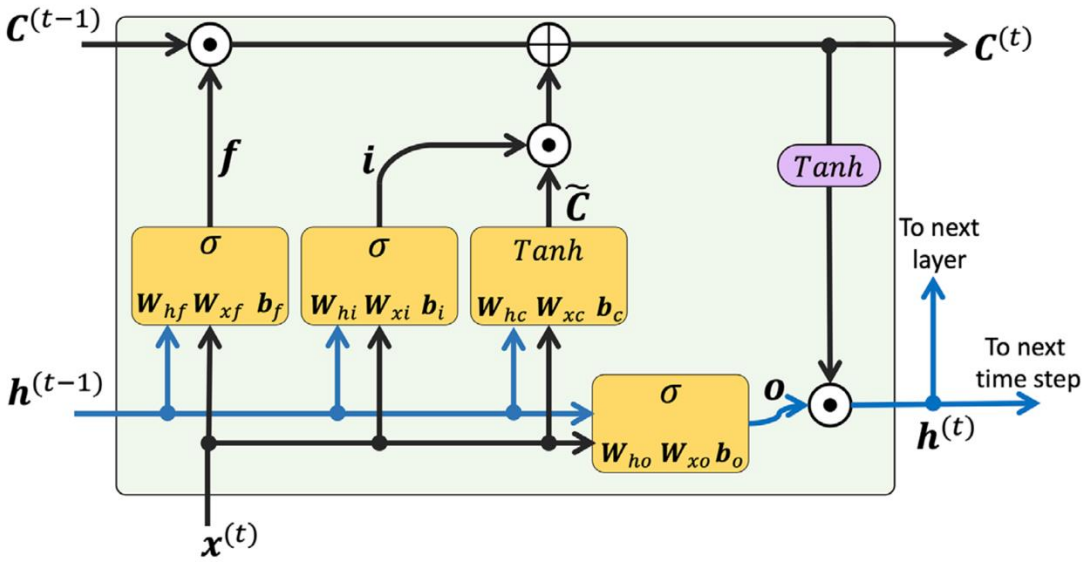
$$i_t = \sigma\left(W_{xi}x^{(t)} + W_{hi}h^{(t-1)} + b_i\right)$$
$$\tilde{C}_t = \tanh\left(W_{xc}x^{(t)} + W_{hc}h^{(t-1)} + b_c\right)$$

The cell state at time t is computed as

$$C^{(t)} = \left(C^{(t-1)} \odot f_t\right) \oplus \left(i_t \odot \tilde{C}_t\right)$$

Element-wise multiplication and summation

Output gate



It decides how to update the values of hidden units

$$\boldsymbol{o}_t = \sigma\left(\boldsymbol{W}_{xo}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{ho}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_o\right)$$

$$\boldsymbol{h}^{(t)} = \boldsymbol{o}_t \odot \tanh\left(\boldsymbol{C}^{(t)}\right)$$

Element-wise multiplication and summation

PyTorch

PyTorch can implement MLP, CNN, RNN, LSTM every effectively.

It is a deep learning framework that automatically handles gradients and optimization, so we can focus on model design instead of calculus.